Team Reflection Journal

Compiled member entries

ITAI 2376 · Creating Images with Diffusion Models

Compiled from supplied member entries. Formatting and completeness checks do not establish authorship, factual correctness, or author approval.

Team Reflection Journal

Michael

What I learned

Building the code and then compiling the team's completed notebooks helped me understand the difference between implementing a model, collecting experimental evidence, and preparing a defensible final report. A pipeline can produce outputs while still requiring checks that the notebook, checkpoint, figures, and written results belong to the same selected experiment.

What I implemented and integrated

I constructed the project code, received the completed notebook sections from my teammates, performed the CLIP integration, and assembled the final deliverables. During final assembly, I brought the technical evidence and the team's written explanations together into one submission.

An issue encountered during final assembly

The PDF-generation process initially produced contribution and reflection journals containing repeated placeholder text instead of the members' actual entries. Finding those placeholders showed me that a successful file-generation command does not establish that a document is complete.

The correction was to make the journal builder read completed member entries and stop when the required text was missing, rather than produce another unfinished document.

What the results demonstrate—and their limits

Training loss measures noise prediction, while requested-class agreement examines whether generated samples match the requested digit according to the evaluator. CLIP adds an image-text alignment experiment. These measurements answer different questions and should remain separate in the assessment.

The final report should distinguish recorded observations from verified correctness. Saved outputs do not by themselves prove clean-runtime reproducibility, and differences between earlier and selected runs should be explained rather than silently overwritten.

What I would improve next time

I would establish the handoff format and journal requirements before distributing the notebook sections. I would also preserve a clearly identified final run, record its artifact references, and collect each member's written entry before generating the submission PDFs.

This would reduce last-minute integration work and make it easier to trace each reported result back to its supporting evidence.

Cherluk

Source: `part1_handoff.json`. Existing statements are preserved as source excerpts; author approval is not inferred.

What i learned

> I learned how the different parts of a U-Net work together for a diffusion
> model. I understand better how the encoder reduces the image size, how skip
> connections keep important information, and how the decoder brings the image
> back to its original size. I also learned how time and class conditioning
> give the model information about the diffusion timestep and which digit it
> is working with.

Hardest part and how i got unstuck

> The hardest part for me was understanding the shapes and channels going
> through the U-Net. I had to follow the shape trace step by step to understand
> why the first UpBlock receives 192 channels after combining 64 channels with
> the 128-channel skip connection. I also had a problem exporting the model
> because the automatic export could not access the source code. I used the
> provided fallback and manually created diffusion_part1.py, then verified that
> it imported correctly and rebuilt a working U-Net.

What i would do differently

> If I did this again, I would keep track of the tensor shapes while building
> each part instead of waiting until the full U-Net was completed. This would
> make it easier to understand where the channels and image sizes change. I
> would also test each block immediately after completing it so problems could
> be found before connecting the entire model together.

Message to olu

> I completed and verified the data pipeline and U-Net components for Part 1.
> The model uses normalized MNIST data, time conditioning, class conditioning,
> and skip connections. The provided checks passed for the U-Net, including
> output shape, conditioning, skip connections, and gradients. The automatic
> export did not work in my notebook, so I used the fallback to create
> diffusion_part1.py manually and verified that the exported file imports and
> rebuilds a working U-Net. You can use diffusion_part1.py as the starting
> point for the next part.

Olu

Source: `part2_handoff.json`. Existing statements are preserved as source excerpts; author approval is not inferred.

What i learned

> I learned that diffusion is easier to understand when I separate it into three jobs: build the noise schedule, add known noise during training, and learn to predict that noise so the reverse process can remove it. The experiments also made the role of the timestep concrete instead of leaving it as just another model input.

Hardest part and how i got unstuck

> The hardest part was keeping the schedule math and tensor shapes straight. I used the printed alpha_bar values, the toy calculation, and the verification checks to isolate mistakes instead of changing several lines at once. The predict_x0 failure was a notebook execution-order problem: the function was correct but had not been defined when the check ran.

What i would do differently

> I would run the notebook strictly from the top after every structural edit and make sure every export tag is present before running the handoff cell. I would also write predictions before running their matching experiments so the comparison remains a genuine prediction rather than a retrospective answer.

Message to taki

> Load best_model.pt and rebuild the schedule with the exact checkpoint settings: 100 steps, beta_start 0.001, beta_end 0.2. Use the UNet from diffusion_part1.py and the sample function from diffusion_part2.py. The model trained well: best validation loss was 0.0230 at epoch 20. The timestep input is important—the wrong-timestep loss was 0.0542 versus 0.0233 with correct inputs—while the wrong class increased loss only slightly to 0.0254. Do not replace the rescaled schedule with the original 0.0001-to-0.02 range, because that 100-step schedule leaves alpha_bar at 0.3636 instead of almost zero. Also rerun the notebook from the top so the predict_x0 verification and forward export are recorded cleanly.

Taki

Source: `/home/ubuntu-user/Documents/MidtermDoc(DeepLearning)/FINAL_SUBMISSION/supporting_files/part3_results.json`.

The following source statements are preserved. Run-dependent percentages and historical CLIP statements require comparison with the selected final experiment; they are not automatically adopted as final-result claims.

Most important finding

> Class conditioning has a real effect but is unreliable. An independent classifier (97.85% on real MNIST) identified the requested digit in only 42% of generated images, versus 10% by chance. Digit 1 reached 100% but 8 reached only 20%. Common confusions were 4 -> 7 (11 of 20), 5 -> 3 (9 of 20) and 6 -> 0 (7 of 20).

Hardest part

> Importing Olu's diffusion_part2.py, which expected training variables (EPOCHS, LR, train_loader, sched) to exist at import time. I set mock the team caught an error with a print staement that since the find, has been deleted the particular block involved in Part 2, providing an unnecessary mock block issued

How i debugged or verified it

> I checked that the checkpoint config and schedule were consistent, then trained a separate small CNN to act as an evaluator and confirmed it scored 97.85% on real MNIST before using it on generated images. I also inspected the confusion matrix instead of relying on the overall number.

What the visualizations show

> In the digit-7 trajectory the image is mostly noise until about t=41, faint structure appears around t=31, strokes are clearer by t=21, and the image is sharp by t=6. The final image does not look like a 7 (it looks closer to a 4 or L shape), which matches the weak class accuracy. With the same starting noise, changing the class changed the output, but not always into the requested digit.

What i learned about conditional generation

> A low training/validation loss (0.023) does not mean the samples follow the requested class. The loss measures noise prediction, so I needed a separate generation-level test to see how well conditioning works.

Clip observation

> I did not run the optional CLIP bonus. CLIP is built for natural images and captions, so tiny grayscale digits are a weak fit, and the classifier-based check was more informative here.

What i would improve

> Try classifier-free guidance, and ask Olu whether more training epochs or a different noise schedule (beta end 0.2 over 100 steps) would improve class accuracy. I have not tested whether either helps.

Message to michael

> All figures, part3_results.json and PART3_HANDOFF.md are ready. Please report the 42% class accuracy honestly and include the confusion matrix. Describe it as the share of 200 generated samples (20 per class) that the independent classifier labeled as the requested digit. Trajectory figures use one convention: the label t=99 is the image entering reverse step 99, and 'final' is the result after the t=0 step.